© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_18

18. Can You See Me Now? Observability for LLM Pipelines

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to understand the role and importance of observability in large language model (LLM) applications and how to set up and measure them in Java applications.

Introduction

In LLM applications, observability is not just a backend concern—it's a critical design principle. With LLMs becoming foundational in modern software systems, understanding how these models behave in real-world scenarios is essential to ensuring performance, reliability, and cost-effectiveness. Observability provides the tooling and practices to make that possible.

At its core, observability in LLM applications means systematically collecting, correlating, and analyzing signals—metrics, traces, and logs—across the AI pipeline. From the time a user makes a request, through the model's inference process, to the final response delivery, each step must be instrumented and made transparent.

This chapter explores how to bring observability into Java-based LLM applications, using tools like Spring AI and LangChain4j. You'll learn how to capture critical performance indicators, monitor token usage, trace end-to-end inference paths, and surface hidden issues that could impact the user experience or operational costs. Whether you're debugging a misbehaving model or optimizing system throughput under load, observability gives you the insight needed to act with confidence.

Why Observability Matters in LLM Evaluation

Effective observability begins by shedding light on performance characteristics across your AI system. When you measure end-to-end invocation times—including client call overhead, network latency, and model inference delays—you gain the insight needed to identify hotspots. Tracking request rates and processing times under varying load levels reveals how your solution scales, allowing you to right-size resources and maintain consistent user experience even as traffic grows.

Cost management in LLM applications often hinges on token consumption and infrastructure utilization. By capturing input and output token counts for each request, you can spot opportunities to refine prompt design and reduce unnecessary generations, directly lowering your billing statement. Similarly, monitoring GPU, CPU, and memory usage during inference provides the visibility required to optimize deployment configurations, tune auto-scaling policies, and avoid surprise overprovisioning that drives up cloud costs.

Reliability is not just about uptime—it's also about being able to diagnose and recover from errors rapidly. When your observability framework records error rates, exception traces, and the contexts in which failures occur (such as malformed prompts or connectivity issues), you can set up alerts for anomalous spikes and jump straight to the root cause. Observability data also validates the effectiveness of retry and fallback strategies by exposing retry counts and end-to-end latencies for secondary code paths.

Understanding model behavior at scale demands visibility into the parameters and content driving each inference. Auditing hyperparameters like temperature, top_k/p, and stop sequences alongside the corresponding outputs makes it possible to correlate specific settings with quality, coherence, or safety outcomes. When responsibly enabled, prompt and completion logs—tightly coupled with trace identifiers—provide a forensic record for investigating hallucinations, bias patterns, or drifts in relevance over time.

In regulated or privacy-sensitive environments, observability plays a crucial role in governance. By associating every inference with a unique trace or conversation ID, you create an auditable lineage of data transformations—a necessity for compliance with data-protection standards. At the same time, observability platforms can be configured to redact or omit sensitive fields from metrics, ensuring telemetry remains actionable without exposing personal or proprietary information.

Core Observability Signals

See Table 18-1.

Table 18-1 Core Observability Signals at a Glance

Signal	Type	Cardinality	Relative Cost	Typical Retention
Metrics	Aggregated numeric data (e.g., counters, histograms, gauges)	Low	Low	Long-term (30–90 days)
Traces	Distributed spans across services with detailed context	High	Medium	Medium (7–30 days)
Logs	Unstructured or semi-structured text (e.g., errors, payloads)	High (Optional)	High (especially if full prompts are included)	Short-term (1–14 days, longer for filtered logs)

Metrics constitute the backbone of lightweight, high-volume telemetry. Low-cardinality metrics—such as histograms for operation durations, counters for success or failure rates, and gauges for token usage—deliver scalable performance insights without overwhelming your monitoring backend. These aggregated numbers answer questions like "What is our 95th percentile latency?" or "How many tokens are we processing per hour?"

Tracing provides the rich, high-cardinality detail that metrics alone cannot capture. Distributed traces stitch together spans from your client libraries, network layers, and model providers, revealing the precise timing breakdown of each request. Within those spans, attributes such as full prompt text, hyperparameter values, and conversation identifiers make it possible to reconstruct exactly how an inference unfolded, enabling deep dives into edge-case failures and performance anomalies.

Logs serve as the final layer of observability for content-rich diagnostics. When controlled by opt-in flags and appropriate retention policies, your logs can include full prompts, model responses, and detailed error contexts. By correlating logs with trace IDs, developers can seamlessly navigate from a high-level metric alert down to the exact input that triggered a failure, speeding up troubleshooting and root-cause analysis.

Best Practices

Separate low- versus high-cardinality data to preserve the performance and usability of your observability stack. Emit only a small, fixed set of labels on your metrics to keep cardinality in check, while reserving detailed, variable attributes—like full prompt text or conversation IDs—for tracing spans. This dual-tiered approach ensures you get both the wildfire visibility of metrics and the forensic depth of traces.

Safeguard sensitive data by default. Configure your observability framework to redact or omit any personal, confidential, or proprietary content from both metrics and spans. Provide clear, documented opt-in controls for teams that require full content logging during debugging, along with guidance on masking or tokenization strategies to maintain compliance with privacy regulations.

Correlate identifiers consistently across components—client SDKs, model backends, embedding services, image generators, and vector stores—so you can reconstruct the full lifecycle of a request. When every span and log entry carries the same unique trace or conversation ID, you unlock end-to-end visibility that cuts across technology silos and infrastructure boundaries.

Automate dashboards and alerts around your key performance indicators. Build visualizations for latency percentiles, error rates, and token consumption trends, and configure threshold-based alerts for sudden deviations. Automated notifications help your team detect regressions or operational issues early, before they impact end users or blow past cost budgets.

Embed observability into your continuous evaluation loop. As you test new model versions or tweak deployment configurations, leverage historical telemetry—such as average inference times, error frequencies, and cost per token—to validate that changes meet your SLOs. By making observability data a core input to your testing and promotion workflows, you ensure only reliable, performant models reach production.

To proactively detect issues before they reach users, teams should automate dashboards and alerts around a set of well-defined key performance indicators (KPIs). These typically include latency (especially P50, P95, and P99 percentiles), error rates (both 4xx and 5xx), token consumption trends, and cost metrics such as cost per thousand tokens or per request. Visualizations like time-series graphs for response latency, heatmaps of error spikes, and histograms of token usage can provide early warning signals that something is wrong—whether due to model drift, traffic surges, or misconfigurations. Alerts should be configured with intelligent thresholds—for instance, triggering a warning when P95 latency exceeds two seconds or when 5xx error rates rise above 1% over a 10-minute window. Token usage spikes can be flagged if they exceed cost budgets for a specific API key or region. By embedding these visual and automated guardrails into your operations, you reduce time-to-detection for regressions and can respond before user experience or cloud bills are negatively impacted.

Integrating Observability in LLMs Through Spring AI

Observability plays a crucial role in working with LLMs, and Spring AI enhances this by leveraging the robust observability features of the Spring ecosystem. It offers detailed insights into AI operations by providing metrics and tracing support for key components such as the ChatClient (including Advisor), ChatModel, EmbeddingModel, ImageModel, and VectorStore.

In the upcoming sections, you will see how to build a generative AI (Gen AI) project from scratch in Spring AI and integrate observability into it at various layers and observe how it helps.

Let's start with going to Spring Initializr and creating a project with the dependencies shown in Figure 18-1. You can always feel free to use any of the existing Spring AI-based projects and integrate the change to it instead. I have chosen a new one to keep it independent and simple to start.

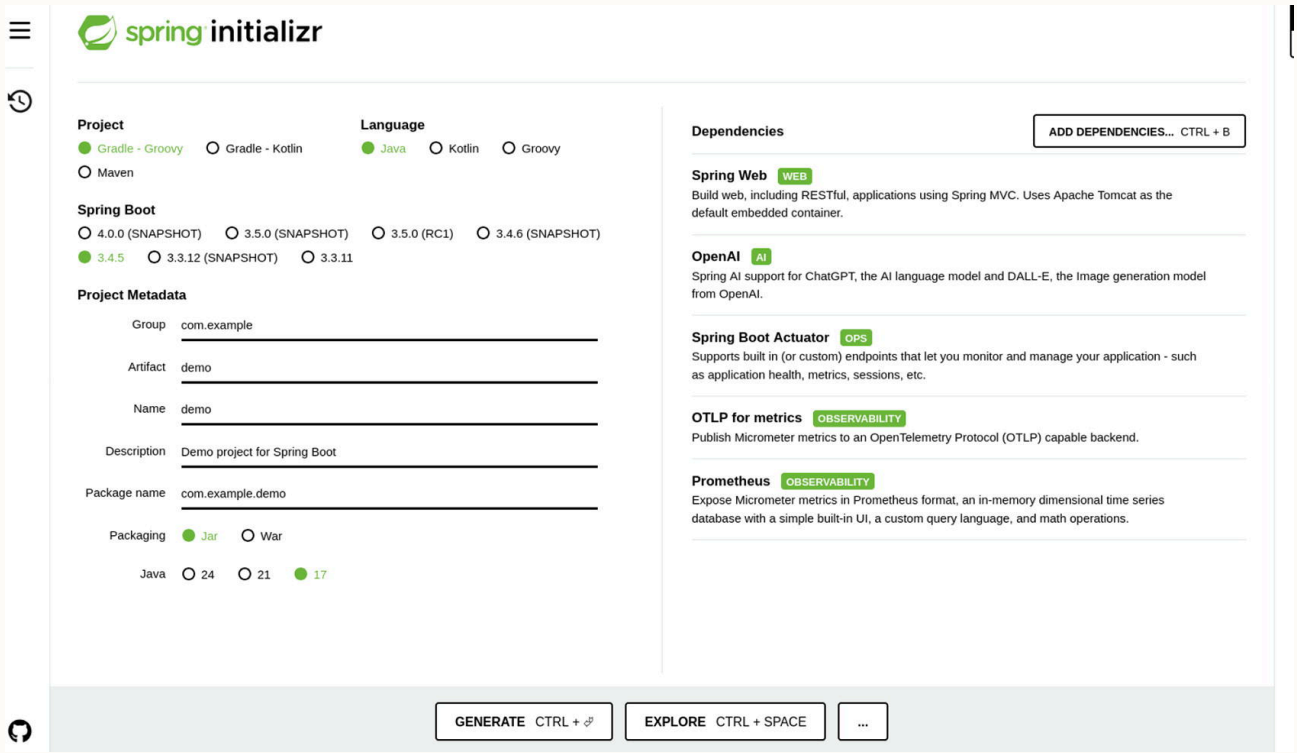


Figure 18-1 Spring Initializr project initialization

First, download it and unzip it at the location of your choice. To make sure everything with the initial setup is okay, you will set one parameter for the Open AI API key in the application.properties file. Also, since you want to capture and see the metrics observed from the Spring AI components, you will set a few other attributes/properties specific to them.

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=YOUR_OPENAI_API_KEY
```

```
management.endpoints.web.exposure.include=*
management.metrics.enable.all=true
management.tracing.enabled=true
```

You will set up a foundation framework to learn about each of the observability settings in Spring AI. All of the settings relate to a particular API call like the use of ChatClient or ChatModel API. So, you will start with a foundation bean method using the CommandLineRunner pattern to run the respective methods when the Spring Boot application is started. You have already seen this pattern in use in previous chapters, so the methods should be self-explanatory.

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
    @Bean
    public CommandLineRunner run() {
        return args -> {
            // Placeholder to make all calls by creating a
            // private method and invoking from here.
        };
    }
}
```

This should set the foundation for the next set of modular components, which we will build around different Spring AI-specific calls like ChatClient, ChatModel, etc. The first one is ChatClient. Let's boot up the application and see the metrics enabled by default at this moment.

```
curl http://localhost:8080/actuator/metrics
```

Output:

{"names":["application.ready.time","application.started.time","disk.free","disk.tota
l","executor.active","executor.completed","executor.pool.core","executor.pool.ma
x","executor.pool.size","executor.queue.remaining","executor.queued","http.server.re
quests","http.server.requests.active","jvm.buffer.count","jvm.buffer.memory.used","j
vm.buffer.total.capacity","jvm.classes.loaded","jvm.classes.unloaded","jvm.compilatio
n.time","jvm.gc.live.data.size","jvm.gc.max.data.size","jvm.gc.memory.allocated","jv
m.gc.memory.promoted","jvm.gc.overhead","jvm.gc.pause","jvm.info","jvm.memory.c
ommitted","jvm.memory.max","jvm.memory.usage.after.gc","jvm.memory.used","jv
m.threads.daemon","jvm.threads.live","jvm.threads.peak","jvm.threads.started","jvm.
threads.states","logback.events","process.cpu.time","process.cpu.usage","process.file
s.max","process.files.open","process.start.time","process.uptime","system.cpu.coun
t","system.cpu.usage","system.load.average.1m","tomcat.sessions.active.current","to
mcat.sessions.active.max","tomcat.sessions.alive.max","tomcat.sessions.created","to
mcat.sessions.expired","tomcat.sessions.rejected"]}

As you can see in the previous output, none of the metrics is relevant to spring.ai components.

Observing ChatClient Requests

Setting the spring.ai.chat.client.observations.log-prompt=true allows you to log observations when a ChatClient call() or stream() method/operation is called. Internally these methods help you measure the time spent by the invocation and share the related tracing instrumentation information for them. Let's see this in practice.

You will update the foundation code to initiate the ChatClient API and make a call request to share a joke. The updated code is as follows:

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
   private ChatClient chatClient;
   public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
   @Bean
```

```
    public CommandLineRunner run(ChatClient.Builder chatClientBuilder) {
        chatClient = chatClientBuilder.build();
        return args -> {
            this.makeChatClientRequest();
        };
    }
    private void makeChatClientRequest() {
        String response = chatClient.prompt()
                .user("Tell me a joke.")
                .call()
                .content();
        System.out.println(response);
    }
}
```

Booting up the application should give you output similar to the following, but you will not see any instrumentation details for the request made to OpenAI.

Output:

```
Why did the scarecrow win an award?
Because he was outstanding in his field!
```

Let's enable the observability parameter in the application.properties file.

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=YOUR_OPENAI_API_KEY
management.endpoints.web.exposure.include=*
management.metrics.enable.all=true
management.tracing.enabled=true
spring.ai.chat.client.observations.log-prompt=true
```

At this moment, in the Spring Boot startup logs, you should see the following info message:

```
2025-05-19T22:21:28.228Z  WARN 11207 --- [demo] [main] .s.a.m.c.c.a.ChatClientAut
oConfiguration : You have enabled logging out the ChatClient prompt content with the
risk of exposing sensitive or private information. Please, be careful!
```

Let's also look at the actuator endpoint for the metrics to see if there are additional metrics being exposed now:

```
curl http://localhost:8080/actuator/metrics
```

Output:


```
{"names":["application.ready.time","application.started.time","disk.free","disk.tota
l","executor.active","executor.completed","executor.pool.core","executor.pool.ma
x","executor.pool.size","executor.queue.remaining","executor.queued","gen_ai.client.
operation","gen_ai.client.operation.active","gen_ai.client.token.usage","http.client.req
uests","http.client.requests.active","http.server.requests.active","jvm.buffer.count","j
vm.buffer.memory.used","jvm.buffer.total.capacity","jvm.classes.loaded","jvm.classe
s.unloaded","jvm.compilation.time","jvm.gc.live.data.size","jvm.gc.max.data.size","jv
m.gc.memory.allocated","jvm.gc.memory.promoted","jvm.gc.overhead","jvm.gc.paus
e","jvm.info","jvm.memory.committed","jvm.memory.max","jvm.memory.usage.after.
gc","jvm.memory.used","jvm.threads.daemon","jvm.threads.live","jvm.threads.pea
k","jvm.threads.started","jvm.threads.states","logback.events","process.cpu.time","pr
ocess.cpu.usage","process.files.max","process.files.open","process.start.time","proces
s.uptime","spring.ai.advisor","spring.ai.advisor.active","spring.ai.chat.client","sprin
g.ai.chat.client.active","system.cpu.count","system.cpu.usage","system.load.average.
1m","tomcat.sessions.active.current","tomcat.sessions.active.max","tomcat.sessions.al
ive.max","tomcat.sessions.created","tomcat.sessions.expired","tomcat.sessions.reject
ed"]}
```


As you can see, additional metrics are being captured for spring.ai.advisor and spring.ai.chat.client. Let's deep dive into the metric spring.ai.chat.client to see the observations it has captured:

```
curl http://localhost:8080/actuator/metrics/spring.ai.chat.client
```

Output:


```
{"name":"spring.ai.chat.client","baseUnit":"seconds","measurements":[{"statistic":"C
OUNT","value":1.0},{"statistic":"TOTAL_TIME","value":1.569205385},{"statistic":"MA
X","value":1.569205385}],"availableTags":[{"tag":"gen_ai.operation.name","values":
["framework"]},{"tag":"spring.ai.kind","values":["chat_client"]},{"tag":"error","value
s":["none"]},{"tag":"spring.ai.chat.client.stream","values":["false"]},{"tag":"gen_ai.sys
tem","values":["spring_ai"]}]}
```


Some of the values that are captured are the name of the observation, which is spring.ai.chat.client; measurements like COUNT stating the number of requests made,

which in our case is 1; and then TOTAL_TIME and MAX time taken. There are a few tag-specific values for gen_ai.operation.name, which will always be framework, and the spring.ai.kind value, which is chat_client for the request in use. This is super helpful to measure the number of requests made and the time it took.

As per the Spring AI documentation page, low cardinality keys will be added to metrics and traces, while high cardinality keys will only be added to traces. Table 18-2 shows the low cardinality keys and values in our metrics endpoint.

Table 18-2 Low Cardinality Keys and Values

Name	Description
genai.operation.name	Always the framework
genai.system	Always spring.ai
spring.ai.chat.client.stream	Is the chat model response a stream, true or false
spring.ai.kind	The kind of framework API in Spring AI: chat_client

To view the high cardinality keys listed in Table 18-3, you will need to set up tracing in Spring Boot. The shared low and high cardinality keys are exposed by default, while the prompt content generally being large in size may contain confidential information that is hidden by default. By setting the parameter to log the prompt as we did previously, you can now capture this information. This may be used for debugging and development use cases but not in production. These are shared in Table 18-3.

Table 18-3 High Cardinality Keys and Values

Name	Description
genai.prompt	The content of the prompt sent via the chat client. Optional.
spring.ai.chat.client.advisors	List of configured chat client advisors.
spring.ai.chat.client.conversation.id	Identifier of the conversation when using the chat memory.
spring.ai.chat.client.tool.names	Names of the tools passed to the chat client.

NoteThe genai.prompt parameter is never populated in the observations and as per GitHub issues, it is rather shown in the debug logs through another attribute. They are yet to update the documentation for this change at this moment of writing.

Let's now work on enabling tracing in Spring Boot so that we can view the high cardinality keys and values.

We will add a few more dependencies to our build.gradle file to enable capturing of the observations.

```
implementation 'io.micrometer:micrometer-core:1.15.0'
implementation 'io.micrometer:micrometer-tracing-bridge-brave:1.5.0'
```

The micrometer dependency package will help us with trace enabling and capturing. These can then be either stored in OpenTelemetry or exported through the Prometheus API service.

File: ./build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.4.5'
    id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
repositories {
    mavenCentral()
}
ext {
    set('springAiVersion', "1.0.0-RC1")
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.ai:spring-ai-starter-model-openai'
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
    implementation 'io.micrometer:micrometer-core:1.15.0'
    implementation 'io.micrometer:micrometer-tracing-bridge-brave:1.5.0'
    runtimeOnly 'io.micrometer:micrometer-registry-otlp'
    runtimeOnly 'io.micrometer:micrometer-registry-prometheus'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
dependencyManagement {
    imports {
        mavenBom "org.springframework.ai:spring-ai-bom:${springAiVersion}"
```

```
    }
}
tasks.named('test') {
    useJUnitPlatform()
}
```

Then you will enable a few more properties in our application.properties file to enable a few endpoints and services like Prometheus and logging for log prompts.

Enable three properties for log prompts at the chat client and chat model levels with the following lines. These three properties seem to have no effect and can be ignored. The future documentation should state useful information relevant to this.

```
spring.ai.chat.observations.log-prompt=true
spring.ai.chat.observations.include-error-logging=true
spring.ai.chat.client.observations.log-prompt=true
```

Enable tracking and prometheus metrics export through the following two properties:

```
management.tracing.enabled=true
management.prometheus.metrics.export.enabled=true
```

Enable logging of log prompt observations through the following two properties:

```
logging.level.org.springframework.ai.chat.observation=DEBUG
logging.level.org.springframework.ai.chat.client.observation=DEBUG
```

All the changes for the application.properties file are shown here:

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=YOUR_OPENAI_API_KEY
spring.ai.chat.observations.log-prompt=true
spring.ai.chat.observations.include-error-logging=true
spring.ai.chat.client.observations.log-prompt=true
management.endpoints.web.exposure.include=*
management.metrics.enable.all=true
management.tracing.enabled=true
management.prometheus.metrics.export.enabled=true
logging.level.org.springframework.ai.chat.observation=DEBUG
```

```
logging.level.org.springframework.ai.chat.client.observation=DEBUG
```

Now that you have enabled the tracing, the high cardinality keys are not observable from the logs except the log prompts. To view them, you have to write a class that implements the ObservationHandler interface as shown next. You have implemented three methods, namely, supportsContext, onStart, and onStop. You can print the observation name and the high cardinality keys and values specific to them by calling context.getName() and context.getHighCardinalityKeyValues().

File: ./src/main/java/com/example/demo/LoggingObservationHandler.java

```
package com.example.demo;
import io.micrometer.observation.Observation;
import io.micrometer.observation.ObservationHandler;
import org.springframework.stereotype.Component;
@Component
public class LoggingObservationHandler implements ObservationHandler<Observatio
n.Context> {
   @Override
   public boolean supportsContext(Observation.Context context) {
      return true;
   }
   @Override
   public void onStart(Observation.Context context) {
      System.out.println("Starting observation: " + context.getName());
      System.out.println(context.getHighCardinalityKeyValues());
      System.out.println("Starting observation: " + context.getAllKeyValues());
   }
   @Override
   public void onStop(Observation.Context context) {
      System.out.println("Completed observation: " + context.getName());
      System.out.println(context.getHighCardinalityKeyValues());
      System.out.println("Completed observation: " + context.getAllKeyValues());
   }
}
```

Let's boot up the application and see the logs in action. In the follwoing logs, I have highlighted the respective observation names captured in the start and stop phases. Observe carefully the high cardinality keys and values just after it. In the third line after it, you can also see all the key values including both low and high cardinality ones. Also, in the logs you can observe in between the user and prompt contexts being shared in the debug context of the properties you had set separately for them.

Output:

Starting observation: **spring.ai.chat.client**
[keyValue(spring.ai.chat.client.advisors=["call"])]
Starting observation: [keyValue(gen_ai.operation.name=framework),keyValue(gen_ai.system=spring_ai),keyValue(spring.ai.chat.client.advisors=["call"]),keyValue(spring.ai.chat.client.stream=false),keyValue(spring.ai.kind=chat_client)]
Starting observation: **spring.ai.advisor**
[keyValue(spring.ai.advisor.order=2147483647)]
Starting observation: [keyValue(gen_ai.operation.name=framework),keyValue(gen_ai.system=spring_ai),keyValue(spring.ai.advisor.name=call),keyValue(spring.ai.advisor.order=2147483647),keyValue(spring.ai.kind=advisor)]
Starting observation: **gen_ai.client.operation**
[keyValue(gen_ai.request.temperature=0.7)]
Starting observation: [keyValue(gen_ai.operation.name=chat),keyValue(gen_ai.request.model=gpt-4o-mini),keyValue(gen_ai.request.temperature=0.7),keyValue(gen_ai.response.model=none),keyValue(gen_ai.system=openai)]
Starting observation: **http.client.requests**
[keyValue(http.url=https://api.openai.com/v1/chat/completions)]
Starting observation: [keyValue(client.name=api.openai.com),keyValue(exception=none),keyValue(http.url=https://api.openai.com/v1/chat/completions),keyValue(method=POST),keyValue(outcome=UNKNOWN),keyValue(status=CLIENT_ERROR),keyValue(uri=/v1/chat/completions)]
Completed observation: **http.client.requests**
[keyValue(http.url=https://api.openai.com/v1/chat/completions)]
Completed observation: [keyValue(client.name=api.openai.com),keyValue(exception=none),keyValue(http.url=https://api.openai.com/v1/chat/completions),keyValue(method=POST),keyValue(outcome=SUCCESS),keyValue(status=200),keyValue(uri=/v1/chat/completions)]
2025-05-20T10:24:45.669Z DEBUG 2731 --- [demo] [main] [682c586c329554d895c49aa2b036a97f-d826b458706b22aa] ChatModelPromptContentObservationHandler : Chat Model Prompt Content:
["You are a professional comedian. Tell a joke.", "Tell me a joke."]
Completed observation: **gen_ai.client.operation**
[keyValue(gen_ai.request.temperature=0.7),keyValue(gen_ai.response.finish_reasons=["STOP"]),keyValue(gen_ai.response.id=chatcmpl-BZERJPPfTRYDDOWBrSzQOprvE1Zv9),keyValue(gen_ai.usage.input_tokens=26),keyValue(gen_ai.usage.output_tokens=18),keyValue(gen_ai.usage.total_tokens=44)]
Completed observation: [keyValue(gen_ai.operation.name=chat),keyValue(gen_ai.request.model=gpt-4o-mini),keyValue(gen_ai.request.temperature=0.7),keyValue(gen_ai.response.finish_reasons=["STOP"]),keyValue(gen_ai.response.id=chatcmpl-BZERJPPfTRYDDOWBrSzQOprvE1Zv9),keyValue(gen_ai.response.model=gpt-4o-mini-2024-07-18),keyValue(gen_ai.system=openai),keyValue(gen_ai.usage.input_tokens=26),keyValue(gen_ai.usage.output_tokens=18),keyValue(gen_ai.usage.total_tokens=44)]
Completed observation: **spring.ai.advisor**
[keyValue(spring.ai.advisor.order=2147483647)]

```
Completed observation: [keyValue(gen_ai.operation.name=framework),keyValue(gen_
ai.system=spring_ai),keyValue(spring.ai.advisor.name=call),keyValue(spring.ai.advisor.
order=2147483647),keyValue(spring.ai.kind=advisor)]
2025-05-20T10:24:45.682Z DEBUG 2731 --- [demo] [main] [682c586c329554d895c
49aa2b036a97f-95c49aa2b036a97f] hatClientPromptContentObservationHandler :
Chat Client Prompt Content:
["user":"Tell me a joke."]
Completed observation: spring.ai.chat.client
[keyValue(spring.ai.chat.client.advisors=["call"])]
Completed observation: [keyValue(gen_ai.operation.name=framework),keyValue(gen_
ai.system=spring_ai),keyValue(spring.ai.chat.client.advisors=["call"]),keyValue(spring.
ai.chat.client.stream=false),keyValue(spring.ai.kind=chat_client)]
Why did the scarecrow win an award?
Because he was outstanding in his field!
```

You can also see this information over the Prometheus metrics endpoint. In the following log outputs, I have shared only Spring AI–specific outputs. You can see you are now capturing various observations relevant to the low and high cardinality keys. These can then be transmitted to any Prometheus observing service to show them in dashboards, visualizations, etc.

```
curl http://localhost:8080/actuator/prometheus
```

Output:

```
34:# HELP gen_ai_client_operation_active_seconds
35:# TYPE gen_ai_client_operation_active_seconds summary
36:gen_ai_client_operation_active_seconds_count{gen_ai_operation_name="chat",gen_ai
_request_model="gpt-4o-mini",gen_ai_response_model="none",gen_ai_system="opena
i"} 0
37:gen_ai_client_operation_active_seconds_sum{gen_ai_operation_name="chat",gen_ai_r
equest_model="gpt-4o-mini",gen_ai_response_model="none",gen_ai_system="openai"}
0.0
38:# HELP gen_ai_client_operation_active_seconds_max
39:# TYPE gen_ai_client_operation_active_seconds_max gauge
40:gen_ai_client_operation_active_seconds_max{gen_ai_operation_name="chat",gen_ai_r
equest_model="gpt-4o-mini",gen_ai_response_model="none",gen_ai_system="openai"}
0.0
41:# HELP gen_ai_client_operation_seconds
42:# TYPE gen_ai_client_operation_seconds summary
43:gen_ai_client_operation_seconds_count{error="none",gen_ai_operation_name="cha
t",gen_ai_request_model="gpt-4o-mini",gen_ai_response_model="gpt-4o-mini-2024-0
7-18",gen_ai_system="openai"} 1
```

44:gen_ai_client_operation_seconds_sum{error="none",gen_ai_operation_name="chat",gen_ai_request_model="gpt-4o-mini",gen_ai_response_model="gpt-4o-mini-2024-07-18",gen_ai_system="openai"} 1.262631313
45:# HELP gen_ai_client_operation_seconds_max
46:# TYPE gen_ai_client_operation_seconds_max gauge
47:gen_ai_client_operation_seconds_max{error="none",gen_ai_operation_name="chat",gen_ai_request_model="gpt-4o-mini",gen_ai_response_model="gpt-4o-mini-2024-07-18",gen_ai_system="openai"} 0.0
48:# HELP gen_ai_client_token_usage_total Measures number of input and output tokens used
49:# TYPE gen_ai_client_token_usage_total counter
50:gen_ai_client_token_usage_total{gen_ai_operation_name="chat",gen_ai_request_model="gpt-4o-mini",gen_ai_response_model="gpt-4o-mini-2024-07-18",gen_ai_system="openai",gen_ai_token_type="input"} 26.0
51:gen_ai_client_token_usage_total{gen_ai_operation_name="chat",gen_ai_request_model="gpt-4o-mini",gen_ai_response_model="gpt-4o-mini-2024-07-18",gen_ai_system="openai",gen_ai_token_type="output"} 18.0
52:gen_ai_client_token_usage_total{gen_ai_operation_name="chat",gen_ai_request_model="gpt-4o-mini",gen_ai_response_model="gpt-4o-mini-2024-07-18",gen_ai_system="openai",gen_ai_token_type="total"} 44.0
204:spring_ai_advisor_active_seconds_count{gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_advisor_name="call",spring_ai_kind="advisor"} 0
205:spring_ai_advisor_active_seconds_sum{gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_advisor_name="call",spring_ai_kind="advisor"} 0.0
208:spring_ai_advisor_active_seconds_max{gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_advisor_name="call",spring_ai_kind="advisor"} 0.0
211:spring_ai_advisor_seconds_count{error="none",gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_advisor_name="call",spring_ai_kind="advisor"} 1
212:spring_ai_advisor_seconds_sum{error="none",gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_advisor_name="call",spring_ai_kind="advisor"} 1.524532722
215:spring_ai_advisor_seconds_max{error="none",gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_advisor_name="call",spring_ai_kind="advisor"} 0.0
218:spring_ai_chat_client_active_seconds_count{gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_chat_client_stream="false",spring_ai_kind="chat_client"} 0
219:spring_ai_chat_client_active_seconds_sum{gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_chat_client_stream="false",spring_ai_kind="chat_client"} 0.0
222:spring_ai_chat_client_active_seconds_max{gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_chat_client_stream="false",spring_ai_kind="chat_client"} 0.0
225:spring_ai_chat_client_seconds_count{error="none",gen_ai_operation_name="framework",gen_ai_system="spring_ai",spring_ai_chat_client_stream="false",spring_ai_kind="chat_client"} 1

```
226:spring_ai_chat_client_seconds_sum{error="none",gen_ai_operation_name="framew
ork",gen_ai_system="spring_ai",spring_ai_chat_client_stream="false",spring_ai_kind="ch
at_client"} 1.537632519
229:spring_ai_chat_client_seconds_max{error="none",gen_ai_operation_name="framew
ork",gen_ai_system="spring_ai",spring_ai_chat_client_stream="false",spring_ai_kind="ch
at_client"} 0.0
```

Since we are currently in the observation section of ChatClient, let's focus on some of the observation values captured.

1. **spring.ai**.chat.client.advisors: You can see the call as one of the advisors in the logs.

```
Starting observation: spring.ai.chat.client [keyValue(spring.ai.chat.client.advisors
=["call"])]
```

2. **spring.ai.chat.client.conversation.id**: This is logged in case of memory-enabled chat.
3. **spring.ai**.chat.client.tool.names: This is not available since we do not have any tools used for our chat client. In case we use it, the tool list should be populated here.

There are useful metrics and observations for the use of the ChatClient API.

Observing Chat Model Requests

Observability features are currently available only for ChatModel implementations from the following AI model providers: Anthropic, Azure OpenAI, Mistral AI, Ollama, OpenAI, Vertex AI, MiniMax, Moonshot, QianFan, and Zhiu AI. Support for additional providers will be added in a future release of Spring AI.

The gen_ai.client.operation observations are captured when the call or stream methods of a ChatModel API are invoked. These observations track the duration of method execution and propagate associated tracing data.

The gen_ai.client.token.usage metrics report the number of input and output tokens consumed by a single model invocation.

Let's integrate the chat model request into the application file, as shown next. I have commented out the call to makeChatClientRequest() so as to focus on the chat model request.

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
   private ChatClient chatClient;
   public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
   @Bean
   public CommandLineRunner run(ChatClient.Builder chatClientBuilder, ChatModel
chatModel) {
      chatClient = chatClientBuilder.build();
      return args -> {
         //makeChatClientRequest();
        makeChatModelRequest(chatModel);
      };
   }
   private void makeChatClientRequest() {
      String response = chatClient.prompt("You are a professional comedian. Tell a jok
e.")
            .user("Tell me a joke.")
            .call()
            .content();
      System.out.println(response);
   }
   private void makeChatModelRequest(ChatModel chatModel) {
      String response = chatModel.call(
            new Prompt("Tell me a joke.")
      ).getResult().getOutput().getText();
      System.out.println(response);
   }
}
```

As per the documentation, Table 18-4 and Table 18-5 show the low and high cardinality keys that are available to capture. The low cardinality keys and values are highlighted in blue (this also includes the high cardinality key values) in the logs, and the high ones are highlighted in green.

Table 18-4 Low Cardinality Keys for Chat Model Request

Name	Description
gen_ai.operation.name	The name of the operation being performed
gen_ai.system	The model provider as identified by the client instrumentation
gen_ai.request.model	The name of the model a request is being made to
gen_ai.response.model	The name of the model that generated the response

Table 18-5 High Cardinality Keys for Chat Model Request

Name	Description
gen_ai.request.frequency_penalty	The frequency penalty setting for the model request.
gen_ai.request.max_tokens	The maximum number of tokens the model generates for a request.
gen_ai.request.presence_penalty	The presence penalty setting for the model request.
gen_ai.request.stop_sequences	List of sequences that the model will use to stop generating further tokens.
gen_ai.request.temperature	The temperature setting for the model request.
gen_ai.request.top_k	The top_k sampling setting for the model request.
gen_ai.request.top_p	The top_p sampling setting for the model request.
gen_ai.response.finish_reasons	Reasons the model stopped generating tokens, corresponding to each generation received.
gen_ai.response.id	The unique identifier for the AI response.
gen_ai.usage.input_tokens	The number of tokens used in the model input (prompt).
gen_ai.usage.output_tokens	The number of tokens used in the model output (completion).
gen_ai.usage.total_tokens	The total number of tokens used in the model exchange.
gen_ai.prompt	The full prompt sent to the model. Optional.
gen_ai.completion	The full response received from the model. Optional.
spring.ai.model.request.tool.names	List of tool definitions provided to the model in the request.

All the keys are instrumentation values that can help you see the usage of the model, see the number of tokens used in input and output, and set metrics around them for alert and auditability purposes. The keys marked in red are not applicable and will be updated in the documentation in the near future per GitHub.

Let's boot up the application and see the logs in action:

Output:

```
Starting observation: gen_ai.client.operation
[keyValue(gen_ai.request.temperature=0.7)]
Starting observation: [keyValue(gen_ai.operation.name=chat),keyValue(gen_ai.request.model=gpt-4o-mini),keyValue(gen_ai.request.temperature=0.7),keyValue(gen_ai.response.model=none),keyValue(gen_ai.system=openai)]
Starting observation: http.client.requests
[keyValue(http.url=https://api.openai.com/v1/chat/completions)]
Starting observation: [keyValue(client.name=api.openai.com),keyValue(exception=none),keyValue(http.url=https://api.openai.com/v1/chat/completions),keyValue(method=POST),keyValue(outcome=UNKNOWN),keyValue(status=CLIENT_ERROR),keyValue(uri=/v1/chat/completions)]
Completed observation: http.client.requests
[keyValue(http.url=https://api.openai.com/v1/chat/completions)]
Completed observation: [keyValue(client.name=api.openai.com),keyValue(exception=none),keyValue(http.url=https://api.openai.com/v1/chat/completions),keyValue(method=POST),keyValue(outcome=SUCCESS),keyValue(status=200),keyValue(uri=/v1/chat/completions)]
2025-05-20T11:21:40.866Z DEBUG 4301 --- [demo] [main] [682c65c3e0d76277870689be18092c9f-870689be18092c9f] ChatModelPromptContentObservationHandler : Chat Model Prompt Content:
["Tell me a joke."]
Completed observation: gen_ai.client.operation
[keyValue(gen_ai.request.temperature=0.7),keyValue(gen_ai.response.finish_reasons=["STOP"]),keyValue(gen_ai.response.id=chatcmpl-BZFKOVpyfkfuOnzxOwfLLJSKCoeaK),keyValue(gen_ai.usage.input_tokens=12),keyValue(gen_ai.usage.output_tokens=18),keyValue(gen_ai.usage.total_tokens=30)]
Completed observation: [keyValue(gen_ai.operation.name=chat),keyValue(gen_ai.request.model=gpt-4o-mini),keyValue(gen_ai.request.temperature=0.7),keyValue(gen_ai.response.finish_reasons=["STOP"]),keyValue(gen_ai.response.id=chatcmpl-BZFKOVpyfkfuOnzxOwfLLJSKCoeaK),keyValue(gen_ai.response.model=gpt-4o-mini-2024-07-18),keyValue(gen_ai.system=openai),keyValue(gen_ai.usage.input_tokens=12),keyValue(gen_ai.usage.output_tokens=18),keyValue(gen_ai.usage.total_tokens=30)]
Why did the scarecrow win an award?
Because he was outstanding in his field!
```

Similarly, there are observation metrics and traces covering low and high cardinality keys for the following APIs that can be captured when the respective APIs are called. I won't go into the details of showing them as they are very self-explanatory and the previous instructions for some of the calls should provide enough guidance.

- Tool calling
- Vector store use
- Image prompt model calls
- Embedding model calls

Conclusion

Observability is a cornerstone of production-grade LLM applications. As this chapter has shown, embedding robust observability into your AI pipeline enables far more than just error tracking—it empowers proactive performance tuning, cost control, compliance assurance, and model quality analysis.

Using Spring AI and LangChain4j, the chapter demonstrated how to instrument key components of an LLM system with detailed metrics and traces. You saw how to monitor ChatClient and ChatModel invocations, track token usage, and capture prompt-level context for debugging. The chapter also covered how to expose these observations through familiar tools like Micrometer, Prometheus, and tracing frameworks to build actionable dashboards and alerts.

Ultimately, observability transforms your LLM system from a black box into a transparent, measurable, and improvable asset. It shortens feedback loops, strengthens reliability, and fosters operational resilience. As LLMs continue to evolve and become more tightly integrated into critical applications, observability will be the guardrail that keeps them performant, predictable, and accountable at scale.

The next chapter covers the current state of machine learning in the Java ecosystem.

Previous chapter

< 17. Talking in Protocols: The MCP Revolution

Next chapter

19. Native-Speed Machine Learning in Java: DJL, ONNX, and JNI >